

S1 — Incident Triage & Resolution

How an incoming MapleSure Insurance incident goes from a raw ticket to a reviewed, ready-to-send resolution — and the reasoning behind each design call along the way.

EPIC STATUS

6 of 7 SCENARIOS.md stories done, plus 5 capabilities built beyond scope

OWNER

Workstream A — data/backend

SPRINT

Build days 0–2, hardened through day 5

DEMO DATE

Fri 24 Jul 2026

UPDATED

20 Jul 2026

Why this scenario exists

L1/L2 support teams lose time in three predictable places: chasing requestors for details a vague ticket never included, re-triaging and re-routing by hand, and hunting for a prior fix by memory or ad-hoc search. None of that is a knowledge problem — it's a *drafting* problem. S1 automates the draft at each of those three points and leaves a support engineer as the one who reviews, edits, and sends.

AI drafts everything; a support engineer reviews, edits if needed, and sends — except two scoped, clearly-labeled exceptions: the quality gate's own requestor info-request email, and self-service password-reset/ID-card requests. Nothing else in this flow is auto-sent or auto-closed.

– demo/presenter_notes/s1_triage.md

That line is the design constraint everything else in this document sits under. It's not a caveat bolted on after the fact — see § **Non-negotiable** for how it's enforced in code, not just the demo script.

Epic → user stories

One story per capability in `SCENARIOS.md`. Status reflects what's actually in the repo, not the plan.

6/7

SCENARIOS.md stories done

2,251

lines across 17 S1 modules

48/48

S1 unit tests passing

7

seeded data patterns exercised

DONE6

BACKLOG1

US-1.0

CORE

Score whether an incoming incident has enough information to investigate

quality_gate.py

US-1.1

CORE

Triage by editable rules first, then AI — AI explains any override

rules_engine.py

triage.py

US-1.2

CORE

Route to the correct assignment group with a citation

routing.py

US-1.6

PLUS

Scan the existing backlog for incidents routed to the wrong group

not started

US-1.3

CORE

Surface similar past incidents and how they were resolved

similar_incidents.py

US-1.4

CORE

Draft a resolution grounded in KB articles and similar-incident history

resolution.py

kb_retrieval.py

US-1.5

PLUS

Free-text Q&A over the incident/KB corpus

chatbot.py

Built beyond SCENARIOS.md scope — added because they earn their place in the live demo narrative, not because a story asked for them:

self_service.py

 auto-resolve for password-reset/ID-card requests

funnel.py

 zero-LLM population forecast

queue_status.py

 live per-ticket status

batch.py

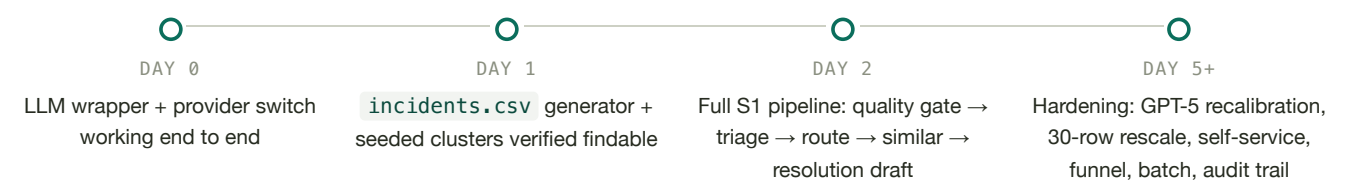
 one-click whole-dataset run

ticket_events.py

 append-only audit trail

03 · SPRINT TIMELINE

Where S1 landed in the build



Per BUILD_PLAN.md, S1 was Workstream A's first full pipeline to reach demo-ready state — S2's problem/RCA flow reuses the same incidents.csv and the same LLM wrapper. What shipped on day 2 kept changing after: the quality-gate prompt was recalibrated live against GPT-5 (2026-07-16), the dataset was rescaled from 600 rows to 30 to support one-click batch runs (2026-07-17), and self-service auto-resolve, the Triage Funnel dashboard, live queue status, batch mode, and the ticket timeline were all added afterward.

04 · WHAT WE BUILT, AND WHY

Capability by capability

1.0 Data-quality gate

quality_gate.py

WHAT

Scores an incoming incident 0–1 on whether it's investigable as-is. If not, drafts a mail back to the requestor asking for the missing specifics — auto-sent, no engineer review.

WHY

This is the differentiator, not just the ask — it stops an engineer from opening a ticket that's going to bounce straight back for more information. Calibrating the boundary took real live-model testing (see § Verification) — a single pass proved nothing near the line.

Self-service auto-resolve

self_service.py

WHAT

Password-reset and digital-ID-card requests matched on category=Access, priority=P4, and keyword text auto-resolve with a fixed template — no LLM call at all.

WHY

The textbook "safe to fully automate" ITSM case: fully reversible, touches only the requestor's own account, no judgment call. Scoped narrow on purpose — "locked out" can just as easily mean a fraud report, so category+priority gates the keyword match.

1.1 Two-level triage — rules, then AI

rules_engine.py + triage.py

WHAT

A plain-text, per-application rule file fires first. Its suggestion is handed to the model as context; the model makes the final call and must explain any override.

WHY

An LLM doesn't know NightlyBatch's heap-growth pattern is a known recurring issue — the client's own team does. Rules encode that as an editable text file the client can change live, with an agreement gate that pauses for a human on deviation or a no-rule verdict.

1.2 Assignment-group routing

routing.py

WHAT

Maps the incident's application to its owning support group and cites how many prior resolved incidents from the same app/category were routed there.

WHY

Deliberately deterministic, no LLM call — this is the highest-traffic beat in any rehearsal. It has to be instant and identical every time; a lookup table plus a scan is the right tool, not a model call.

1.3 Similar-incident retrieval

similar_incidents.py

WHAT

TF-IDF cosine similarity over application, category, description, and known-error/problem linkage — ranked and returned with resolution notes attached, floor 0.05.

WHY

Also deterministic by design — "seen 4 times before, here's what fixed it" has to be a fact you can trust, not a model paraphrase of one. Retrieval earns that trust; generation doesn't need to be involved until the next step.

1.4 RAG resolution draft

resolution.py + kb_retrieval.py

WHAT

Grounds a drafted resolution — steps, work note, requestor email — in KB chunks and similar-incident notes actually retrieved, not general model knowledge.

WHY

The highest-stakes output in S1: it's the one closest to reaching a requestor. Grounding it in retrieved, real prior resolutions keeps the draft anchored to what actually worked; the prompt itself forbids the model from claiming anything was sent or applied.

1.5 Chatbot mode

chatbot.py

WHAT

Free-text Q&A over the same incident and KB corpus, alongside the structured auto-draft flow.

WHY

Not every question an engineer has fits the six-beat pipeline shape. Built cheaply because the retrieval helpers already existed for 1.3/1.4 — exposing them as open Q&A cost little extra.

Triage Funnel & live queue status

funnel.py + queue_status.py

WHAT

Forecasts, for the whole population and with zero LLM calls, how many incidents would even need a human to figure out what they are — then a live view overrides the forecast once a ticket is actually opened.

WHY

The opening screen of the demo. Calling the real quality gate per incident just to render a dashboard would mean dozens of live calls on every page load — the rule engine and similarity score already answer the forecast question for free.

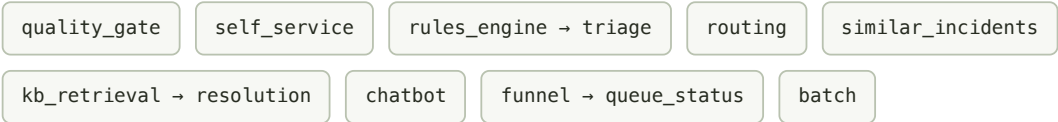
How the pieces fit together

Every LLM call funnels through one module. Nothing else in S1 knows which provider is configured.

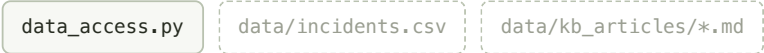
PRESENTATION

s1_triage/app.py – Streamlit

S1 PIPELINE MODULES



DATA ACCESS



COMMON



DECISION	REASONING
All LLM access through one module	No S1 file imports anthropic or openai directly except llm.py — swapping LLM_PROVIDER changes zero pipeline code.
Rules run before any LLM call	Per-application rule files are plain key: value text a client engineer can edit without a code change — validated loudly on load, not silently ignored.
Routing, similarity, and the funnel are deterministic	No LLM call, no cache, no flakiness risk on the beats most likely to be re-run live during rehearsal or clicked repeatedly on stage.
Self-service and quality-gate exceptions are named, not implicit	Two labels (AUTO_SENT_LABEL , AUTO_RESOLVED_LABEL) exist specifically so the two automated paths never read as "the AI acts unsupervised" by default.

One incident, eight beats

What `app.py`'s Auto-draft mode runs, in order, for the engineer's selected incident.

- 1 Self-service check**
`match_self_service()` checks category, priority, and keyword text before anything else runs.
 No LLM
- 2 Quality gate**
 Incident text → `complete()`, cached per incident number → investigable verdict, score, and a drafted requestor mail if it isn't.
 LLM call
- 3 Rule-first, AI-final triage**
`rules_engine.apply_rules()` checks the app's rule file with no LLM involved, then its output is handed to the model as context for the final category/priority call.
 Rules + LLM call
- 4 Agreement gate**
 Agree auto-proceeds. Deviation or AI-only (no rule fired) pauses the pipeline for the engineer to arbitrate.
 Human action, conditional
- 5 Assignment-group routing**
 App→group lookup plus a precedent count scanned from `incidents.csv` — fully deterministic.
 No LLM
- 6 Similar-incident retrieval**
 TF-IDF cosine similarity ranks prior incidents, resolution notes attached.
 No LLM
- 7 RAG resolution draft**
 Relevant KB chunks selected by TF-IDF, combined with the similar-incident notes, and handed to the model for a grounded draft.
 LLM call
- 8 Review & approve**
 Engineer clicks "Mark as reviewed & approved" — a session-only flag plus a timeline event. No API call, no ticket update, no email sent.
 Human action

If any step's LLM call fails, `app.py` catches it once around the whole block, shows a warning, and leaves the already-rendered incident header intact — Streamlit reruns the full script on every interaction, so an unhandled exception would blank the page instead.

Human-in-the-loop

AI suggestion – verify with your specialist before applying.

This exact string (`common/constants.py:AI_SUGGESTION_LABEL`) precedes every AI output rendered in `app.py` . It isn't a UI convention layered on top — `quality_gate.render()` and `resolution.render()` both hard-code language that the draft is for engineer review, and the resolution prompt itself instructs the model to never state that anything was sent or applied. The flow's contract ends at *resolved + ticket updated*; closure is explicitly the client's side, not ours.

Two named, narrowly-scoped exceptions exist and are labeled distinctly so neither generalizes into "the AI acts unsupervised": the quality gate's own info-request mail (`AUTO_SENT_LABEL` — a request for information, never a resolution or ticket action) and the self-service confirmation (`AUTO_RESOLVED_LABEL` — fully reversible, account-scoped, zero AI judgment involved).

What S1 reads, and what it produces

Shared (owned by `common/schema.py` , S1 never writes to it)

TYPE	SOURCE	CONSUMED BY
<code>Incident</code>	Generated by <code>datagen/generate_incidents.py</code> into <code>data/incidents.csv</code>	S1, S2, S6
<code>KbArticle</code>	One dataclass per file under <code>data/kb_articles/*.md</code>	S1, S4

S1-internal (each function is a pure `(inputs) → dataclass`)

TYPE	MODULE	FIELDS
<code>QualityVerdict</code>	<code>quality_gate.py</code>	<code>investigable</code> , <code>score</code> , <code>missing_info</code> , <code>reasoning</code> , <code>drafted_requestor_mail</code>
<code>SelfServiceMatch</code>	<code>self_service.py</code>	<code>kind</code> , <code>confirmation</code>
<code>RuleMatch</code>	<code>rules_engine.py</code>	<code>rule_name</code> , <code>suggested_category</code> , <code>suggested_priority</code> , <code>why</code>
<code>TriageResult</code>	<code>triage.py</code>	<code>rule_match</code> , <code>ai_category</code> , <code>ai_priority</code> , <code>ai_reasoning</code> , <code>agreement</code> , <code>final_category</code> , <code>final_priority</code>
<code>RoutingResult</code>	<code>routing.py</code>	<code>assignment_group</code> , <code>citation</code>
<code>SimilarIncident</code>	<code>similar_incidents.py</code>	<code>incident_number</code> , <code>similarity_score</code> , <code>resolution_notes</code>
<code>ResolutionDraft</code>	<code>resolution.py</code>	<code>resolution_steps</code> , <code>work_note</code> , <code>requestor_email</code>
<code>FunnelClassification</code>	<code>funnel.py</code>	<code>incident_number</code> , <code>stage</code> , <code>rule_matched</code> , <code>best_similarity_score</code> , <code>reasoning</code>
<code>BatchResult</code>	<code>batch.py</code>	<code>incident_number</code> , <code>note</code>

Testing & seeded-data coverage

Deterministic logic is tested for real against `incidents.csv` ; LLM-calling logic is tested with `common.llm.complete` mocked at the call site, asserting on the constructed prompt and the parsed result.

SEED	ROWS	S1 CAPABILITY EXERCISED
<code>cluster_a_memory_leak</code>	5 (NightlyBatch)	1.1 — rule fires (Cluster A heap growth → Batch/P2), AI agrees with reasoning
<code>cluster_b_sso_timeout</code>	5 (PolicyCore/ClaimsPortal)	1.3 — similar-incident retrieval
<code>cluster_c_claims_upload_failure</code>	5	1.3 similar-incident retrieval, 1.4 RAG resolution
<code>misrouted_ticket</code>	2 incl. INC000019	1.2 — routing recommends the correct group despite the seeded misroute
<code>junk_quality_gate</code>	3	1.0 — quality gate rejects and drafts a requestor mail
<code>self_service_password_reset</code>	5 (PolicyCore)	self-service auto-resolve, no LLM call at all
<code>self_service_id_card_resend</code>	5 (PolicyCore)	self-service auto-resolve, second matcher entry

Live-model verification — 2026-07-20

`gpt-5 · tools/verify_s1_live.py · 8/8 checks passed`

Re-ran the project's own live-verification harness against the configured OpenAI key rather than trust the last calibration note in `datagen/SEEDS.md` . It multi-trials the quality gate 5x live per case — a single pass isn't evidence near this boundary, which is exactly how the SEEDS.md row-57 flakiness bug hid from a one-shot check on 2026-07-16. Result: the seeded junk row rejected 5/5, the clean demo row accepted 5/5, and all six background templates accepted 5/5 — 45 live calls, zero flips. The cache-hit-on-rerun check also passed: a repeat call against an already-cached incident added zero new cache files.

`python -m datagen.verify_seeds` → `PASS: seed verification succeeded for data/incidents.csv` . `pytest tests/test_s1_*.py` → **48 passed** across quality gate, triage, routing, similar incidents, self-service, KB retrieval, funnel, review gate, warm cache, and the seeded demo rows.

Live UI verification — 2026-07-20

`Real browser · Chromium via Playwright · live GPT-5 calls`

Not just the CLI quality-gate slice — the actual `demo/run_s1.sh` Streamlit app, driven end to end in a real browser against the live model. Screenshots below are from that run, unedited except for cropping.

✓ Triage Funnel	Renders 30/16/11/3 split live, no LLM call, from the rule engine and similarity scores
✓ Rule + AI agree	INC000001 — rule and model both land on Batch/P2 with stated reasoning; auto-triaged
✓ Quality gate reject	INC000016 ("app not working") scored 0.20, correctly rejected, pipeline halts before triage
✓ Self-service	INC000021 auto-resolved with no engineer review; queue status flips to "AI-resolved"

- ✓

Chatbot

Free-text question answered and grounded, citing KE0000456 / PRB0001234
- ✓

Resolution draft

Steps / work note / requestor email all populate, plus a live ticket timeline

MAPLESURE INSURANCE - AMS CONSOLE - 51

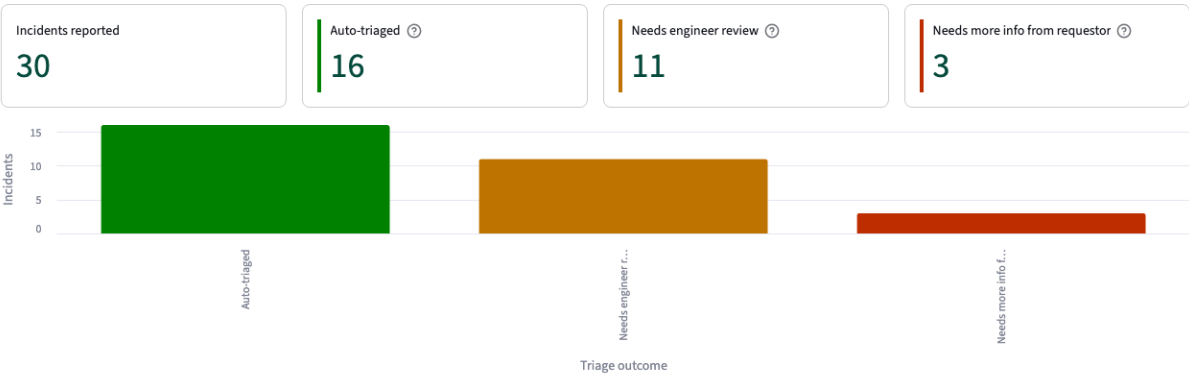
Incident Triage & Resolution

Data-quality gate, rule-first + AI triage, routing, similar-incident retrieval, and RAG resolution drafting — a support engineer reviews every output before it reaches a ticket, except the quality gate's own info-request email to the requestor, which is auto-sent.

Mode

- Triage Funnel
- Auto-draft mode
- Batch mode
- Chatbot mode

Deterministic — based on the editable rule engine and Knowledge Repository (past-incident) match confidence, not an LLM call. The AI-drafted resolution for an "Auto-triaged" ticket still needs an engineer's sign-off before anything is sent — this only shows how many tickets don't need a human to figure out *what* they are.



Triage Funnel, default view — 30 incidents, 16 auto-triaged / 11 need engineer review / 3 need more info from the requestor. Zero LLM calls to render this.

INC000001: NightlyBatch heap growth stalls policy extract

NightlyBatch | Batch/Memory Leak | Priority P2

✔ Auto-triaged — Rule 'Cluster A NightlyBatch heap growth' matched; top Knowledge Repository match score 1.00.

AI suggestion — verify with your specialist before applying.

Investigable	Score	It names the NightlyBatch component in Prod (nightly-worker-01), specifies the symptom (heap growth/allocation pressure causing the batch window to stall), and gives timing and duration (~02:00 ET for ~45 minutes). This is sufficient for an engineer to start investigating with logs and monitoring.
Yes	1.00	

Rule match

Cluster A NightlyBatch heap growth — category: Batch, priority: P2

Matches the Cluster A NightlyBatch heap-growth pattern linked to KE0000456 and PRB0001234. Matched keywords: memory, heap, alloc, worker.

AI suggestion — verify with your specialist before applying.

AI category	AI priority	Agreed with the rule. Prod NightlyBatch heap growth stalled the policy extract and delayed the morning reporting feed, a significant degradation warranting P2 but not a total outage.
Batch	P2	

✔ Rule & AI agree (P2 / Batch) — auto-triaged, no engineer review needed.

Suggested assignment

Full pipeline, rule & AI agree — INC000001 (Cluster A NightlyBatch heap growth): quality gate → rule match → AI triage, live GPT-5 reasoning shown verbatim.

Cluster A NightlyBatch heap growth — category: Batch, priority: P2

Matches the Cluster A NightlyBatch heap-growth pattern linked to KE0000456 and PRB0001234. Matched keywords: memory, heap, alloc, worker.

AI suggestion — verify with your specialist before applying.

AI category

Batch

AI priority

P2

Agreed with the rule. Prod NightlyBatch heap growth stalled the policy extract and delayed the morning reporting feed, a significant degradation warranting P2 but not a total outage.

✓ Rule & AI agree (P2 / Batch) — auto-triaged, no engineer review needed.

Suggested assignment

Assignment group: Batch Ops

Application NightlyBatch maps to Batch Ops; similar to 4 prior resolved NightlyBatch/Batch incidents routed there; Final triage: Batch / P2; rule cited: Cluster A NightlyBatch heap growth.

Similar incidents

incident_number	similarity_score	resolution_notes
INC000005		1 Applied known workaround from KE0000456: restart service, rerun batch after clearing the stuck queue marker. Lir
INC000004		1 Applied known workaround from KE0000456: restart service, rerun batch after clearing the stuck queue marker. Lir
INC000003		1 Applied known workaround from KE0000456: restart service, rerun batch after clearing the stuck queue marker. Lir
INC000002		1 Applied known workaround from KE0000456: restart service, rerun batch after clearing the stuck queue marker. Lir

AI suggestion — verify with your specialist before applying.

Resolution steps

- 1) Confirm impact and scope:
- Environment: Prod; host: nightly-worker-01; application: NightlyBatch.

- Verify batch window status and whether policy extract is stalled (no window advancement since ~02:00 ET).

- Check downstream impact to morning reporting schedule and note any SLAs at risk.

Work note

Draft for engineer review before updating: Observed Prod NightlyBatch stall on nightly-worker-01 with heap growth and allocation pressure starting ~02:00 ET for ~45 minutes, consistent with KE0000456 and prior incidents (INC000002-INC000005 linked to PRB0001234). Prepared plan to capture diagnostics, clear the stuck queue marker, restart the NightlyBatch service on the affected node, and rerun the stalled policy extract window. Will coordinate change control as needed, notify reporting stakeholders of brief interruption risk, and monitor for batch vs. prod downstream feedback. All actions, timestamps, logs, and results to be documented and linked to KE0000456 and PRB0001234. Engineer will review.

Requestor email

Subject: INC000001 – NightlyBatch policy extract delay and recovery plan (Prod)

Hello,

Mark as reviewed & approved

Routing, similar incidents, and the resolution draft — same incident, continued: citation-backed assignment group, four similar prior incidents, and a live-drafted resolution with a real ticket timeline below it.

INC000016: app not working

PolicyCore | Inquiry/Insufficient Details | Priority P4

⚠ Needs more info from requestor — Description too thin to match against — needs more detail from the requestor.

AI suggestion — verify with your specialist before applying.

Investigable

No

Score

0.20

The ticket only names the application (PolicyCore) but lacks a specific symptom and any timing or scope, so an engineer would be guessing what to investigate.

Missing: specific symptom or error message, approximate time/time window, scope/impact (one user vs multiple, specific policy/case if applicable), environment (Prod/UAT), component/page or action within PolicyCore and steps to reproduce

Quality gate: not investigable as-is. Pipeline stops here — triage, routing, similar-incident lookup, and resolution drafting do not run against an incident that failed this gate.

Quality gate stops the pipeline — INC000016 ("app not working"), scored 0.20 and rejected live. Triage, routing, similar-incident lookup, and resolution drafting correctly do not run.

INC000021: Forgot my password, cannot log in

PolicyCore | Access/Password Reset | Priority P4

⚠ Needs engineer review — No rule matched; top Knowledge Repository match score 1.00.

AI auto-resolved — self-service request (no engineer review; demo exception).

Self-service auto-resolve — INC000021 ("Forgot my password"), matched and auto-resolved with no engineer review — the one case where the queue status "AI-resolved" is literally true, not a forecast.

Ask about incident history or KB articles

What caused the NightlyBatch heap growth incidents and how were they fixed?

AI suggestion — verify with your specialist before applying.

Summary

- Cause: All five incidents (INC000001–INC000005) point to steady heap growth/memory leak in the NightlyBatch workers during the policy extract window, which led to the batch stalling. This aligns with KE0000456. Exact root cause details are not in the provided data; PRB0001234 is linked, suggesting an active known problem investigation.
- Fix applied: The temporary workaround from KE0000456 was used in each case—restart the NightlyBatch service, clear the stuck queue marker, and rerun the batch. This restored processing.

Draft for engineer review (recommendation)

- If the stall recurs, apply KE0000456's workaround (restart service, clear stuck queue marker, rerun batch) and track PRB0001234 for the permanent fix. Monitoring heap during the policy extract window is advised to confirm recurrence and timing.

Chatbot mode — free-text Q&A grounded in the same incident/KB corpus, labeled as an AI suggestion for engineer review.

10 • RISKS

What could go wrong, and the mitigation

RISK	MITIGATION
● Quality gate is not fully deterministic near its calibration boundary	Documented live on 2026-07-16 (datagen/SEEDS.md): the same template returned different verdicts across 3 live calls. Fix was to loosen the prompt's evidence bar, not just add examples — re-verify with 5x-trial checks, never a single pass, after any prompt change.
● Shared <code>.cache/llm</code> wipes every scenario's cache on reset	Acceptable while S1 is the heaviest cache consumer; revisit with a scenario-scoped <code>cache_key</code> prefix once S2/S3/S5/S6 all cache as heavily.
● Rule file edited incorrectly live in front of the client	<code>rules_engine._parse_rule_file</code> raises immediately on a malformed line — rehearse the edit beforehand so the failure mode is known, not discovered live.

● Client asks for misroute-scanning (US-1.6) live	It's PLUS-priority and unbuilt, roadmap-only — say so, don't improvise it on stage.
● Batch mode can't pause for arbitration	By design — a headless run has no UI to pause into. A live deviation/AI-only case is flagged in its result note for follow-up instead of gated.

`SCENARIOS.md` — full capability decomposition, all six scenarios

`BUILD_PLAN.md` — 7-day schedule and workstream split

`datagen/SEEDS.md` — seeded acceptance contract for incidents.csv, including live-calibration history

`demo/presenter_notes/s1_triage.md` — live demo script

`tools/verify_s1_live.py` — the live-verification harness re-run for § Verification